

PropControl Unified Billing, Credits, and Schema Master Build Package

Founder-ready source of truth for subscriptions, prepaid credits, Stripe, usage metering, Market Swarm, Vendor Swarm, and the full production database schema appendix

Purpose	Single package for your emergent AI agent and engineering team so nothing has to be pieced together from multiple chats.
Business model	Base subscription tiers for platform access plus prepaid credits for all premium AI, data, swarm, and lead-gen usage.
System of record	Supabase/Postgres is the source of truth for balances, ledgers, purchase orders, usage logs, swarm tracking, and webhook state.
Payment rail	Stripe Checkout now, off-session auto-reload later, with webhook-only credit grants.

Document Map

1. Executive decision summary
 2. Final product and pricing model
 3. Production build sequence
 4. Stripe purchase and webhook sequence
 5. Usage metering and billing middleware
 6. Swarm cost governance for Market Swarm and Vendor Swarm
 7. UI, admin, alerts, QA, and launch controls
- Appendix A. Full database schema appendix
- Appendix B. Core RPC/database functions
- Appendix C. Implementation notes for the AI agent

1. Executive decision summary

- Do not switch to credits only. Keep Growth, Pro, and Team as subscription plans for platform access, then charge prepaid credits for all expensive AI and data actions.

- Treat Alex A.I., Neural Predictor, Visual SOW, RentCast pulls, Market Swarm, Vendor Swarm, Street View qualification, Places discovery, bulk work order analysis, appraisal generation, and exports as premium credit-gated actions.
- Do not use Stripe as the balance source of truth. Stripe confirms money movement; the database controls balances, ledgers, purchase orders, usage logs, swarm runs, and webhook states.
- Do not expose raw provider pricing to users. Convert blended workflow cost into simple action-based credits.
- Do not grant credits from the success page. Credits are granted only after a verified Stripe webhook inserts an idempotent deposit ledger entry.

2. Final product and pricing model

Included in base subscription tiers

- Dashboard access, CRM, workflows, saved properties, team controls, notifications, and standard app usage.
- Base support and normal product navigation.

Charged in credits

- Alex A.I. conversation turns
- Neural Predictor reports
- Visual SOW and any Gemini Vision or Street View-assisted property review
- RentCast market pulls and comps enrichment
- Market Swarm lead-generation runs
- Vendor Swarm vendor discovery runs
- Bulk work order analysis, exports, appraisal generation, and underwriting outputs

Launch credit packs

- \$10 = 1,000 credits
- \$25 = 2,750 credits
- \$50 = 6,000 credits

Pricing principle

- Price by action outcome, not by exposed token math. Internally you calculate wholesale cost across providers, apply markup, and convert to credits. Externally the user sees simple actions like Market Swarm Scan - 120 credits.

3. Production build sequence

Phase 1 - economics and governance

- Finalize which features stay included versus which are credit-gated.
- Create a feature pricing matrix with feature name, providers touched, average requests/tokens, average wholesale cost, target markup, and credits charged.
- Set hard limits for Market Swarm and Vendor Swarm: default batch size, max radius, max results, and daily run caps.

Phase 2 - database foundation

- Run the unified schema migration in Appendix A.
- Seed pricing_configs and credit_packages.
- Verify row-level security and service-role-only paths.

Phase 3 - Stripe purchase flow

- Create internal credit_purchase_orders before Stripe Checkout sessions.
- Implement webhook-only credit grant flow using stripe_webhook_events and deposit_credits().
- Display success and cancel screens, but never grant credits client-side.

Phase 4 - usage metering

- Wrap Gemini, Kimi, RentCast, Places, Street View, and other billable providers behind one common server-side billing middleware.
- Write api_usage_logs and deduct credits only through an atomic database function.
- Add reserve-then-settle for expensive swarm workflows if needed.

Phase 5 - UX and admin tooling

- Credit balance widget, usage history, buy-credits page, low-balance modal, auto-reload settings, admin billing dashboard, webhook event viewer, and manual adjustments.

Phase 6 - launch controls

- Add alerts for failed webhooks, duplicate events, stuck purchase orders, impossible balances, missing deductions, and abnormal swarm spend.
- Soft launch to limited users, review margin by feature, then open wider.

4. Stripe purchase and webhook sequence

1. User selects a credit pack.
2. Backend creates a credit_purchase_orders row first.

3. Backend creates Stripe Checkout Session with `purchase_order_id`, `user_id`, `credits_to_grant`, and package metadata.
4. User pays in Stripe Checkout.
5. Stripe sends `checkout.session.completed` or `async success` event.
6. Webhook verifies the signature from the raw request body.
7. Webhook writes `stripe_webhook_events` if the event is new.
8. Webhook marks the event processing, then calls `deposit_credits()` with an idempotency key tied to the Stripe event.
9. Webhook marks the purchase order paid, stores Stripe identifiers, optionally stores payment method for future auto-reload, then marks the webhook processed.
10. Frontend reads the updated balance from `users.available_credits` and the history from `credit_ledger`.

Never break these rules

- Never grant credits from the success redirect.
- Never process the same Stripe event twice.
- Never update balance without an immutable ledger row.
- Never trust event order. Always re-check internal order state.

5. Usage metering and billing middleware

- Every premium action must hit a single server-side billing wrapper.
- The wrapper identifies the user, feature, and workflow, loads active pricing config, checks balance, executes provider calls, computes wholesale and retail cost, writes `api_usage_logs`, deducts credits, then evaluates low-balance or auto-reload logic.
- For heavy workflows, estimate the run cost before execution and show the estimated credits in the UI.
- Only successful billable actions should be charged unless you intentionally bake retries into your pricing model.

6. Swarm cost governance for Market Swarm and Vendor Swarm

- Market Swarm and Vendor Swarm must be treated as premium, credit-gated lead-generation workflows.

- Each swarm run should have a parent record in `swarm_runs` and child provider-step records in `swarm_run_steps`.
- Track places requests, street view requests, geocoding requests, leads found, qualified leads, and actual blended cost per run.
- Cap expensive runs with batch limits, result caps, and estimated-credit warnings before execution.
- Expose simple action-based pricing to the user while preserving internal provider-level observability.

Recommended swarm safeguards

- Default preview mode before a full lead-gen run.
- Per-run max results and hard stop limits.
- Daily and monthly spend caps by user and by workspace.
- Reserve-then-settle credit option for long-running or multi-step swarm workflows.

7. UI, admin, alerts, QA, and launch controls

Required user-facing UX

- Dashboard credit balance widget
- Buy credits page with package cards
- Transaction and usage history
- Low-balance banner and insufficient-balance modal
- Estimated action cost labels for expensive runs
- Auto-reload preferences after the core flow is stable

Required founder/admin tooling

- Balance viewer, ledger viewer, usage viewer, purchase orders, webhook log, failed webhook retry flow, manual adjustment tool, and balance recalculation action.
- Margin dashboard by feature, provider, and swarm type.

QA before launch

- Duplicate webhook replay test
- Success redirect with missing webhook test
- Duplicate deduction test
- Rapid repeated click test

- Insufficient balance test
- Swarm large-batch cost control test
- Balance recalc from ledger test

Appendix A. Full Database Schema Appendix

Schema overview

Table / Function	Purpose	Key note
users	Cached balance and Stripe linkage	Fast UI reads only; ledger remains source of truth
pricing_configs	Provider cost and markup config	Never hardcode pricing
credit_packages	One-time prepaid packs	Supports promos without deploy
credit_purchase_orders	Internal order before Stripe	Webhook fulfillment maps back here
api_usage_logs	Raw billable provider usage	Per request or per feature execution
credit_ledger	Immutable money and credit history	Every balance movement gets a row
stripe_webhook_events	Webhook dedupe and audit	Prevents double credit grants
swarm_runs	Parent record for Market/Vendor Swarm execution	Tracks estimated and actual blended cost
swarm_run_steps	Provider-level steps inside a swarm	Shows where spend actually came from
swarm_leads	Optional lead-level traceability	Useful for cost per lead analysis

Unified schema SQL

```
create extension if not exists pgcrypto;

alter table public.users
  add column if not exists stripe_customer_id text,
  add column if not exists stripe_payment_method_id text,
  add column if not exists available_credits numeric(18,6) not null default 0,
  add column if not exists auto_reload_enabled boolean not null default false,
  add column if not exists auto_reload_amount numeric(10,2) not null default 10.00,
  add column if not exists low_balance_threshold numeric(10,2) not null default 2.00;

create index if not exists idx_users_stripe_customer_id
  on public.users (stripe_customer_id);

create table if not exists public.pricing_configs (
  id uuid primary key default gen_random_uuid(),
  provider text not null,
  model_name text,
  cost_per_1k_input numeric(18,8) not null default 0,
  cost_per_1k_output numeric(18,8) not null default 0,
  flat_fee_per_request numeric(18,8) not null default 0,
  markup_multiplier numeric(8,4) not null default 2.5,
  is_active boolean not null default true,
  notes text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create index if not exists idx_pricing_configs_provider_model
```

```

    on public.pricing_configs (provider, model_name);

create table if not exists public.credit_packages (
    id uuid primary key default gen_random_uuid(),
    code text not null unique,
    name text not null,
    price_usd numeric(10,2) not null,
    credits_granted numeric(18,6) not null,
    is_active boolean not null default true,
    sort_order integer not null default 100,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    constraint chk_credit_packages_price_nonnegative check (price_usd >= 0),
    constraint chk_credit_packages_credits_positive check (credits_granted > 0)
);

create table if not exists public.credit_purchase_orders (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.users(id) on delete cascade,
    package_id uuid not null references public.credit_packages(id),
    package_code text not null,
    package_name text not null,
    price_usd numeric(10,2) not null,
    credits_to_grant numeric(18,6) not null,
    currency text not null default 'usd',
    status text not null default 'created',
    stripe_checkout_session_id text unique,
    stripe_payment_intent_id text unique,
    stripe_invoice_id text unique,
    stripe_charge_id text,
    client_reference_id text not null unique,
    request_idempotency_key text not null unique,
    paid_at timestamptz,
    created_at timestamptz not null default now(),
    updated_at timestamptz not null default now(),
    constraint chk_purchase_order_status check (
        status in ('created', 'checkout_open', 'paid', 'failed', 'canceled', 'refunded')
    )
);

create index if not exists idx_credit_purchase_orders_user_id
    on public.credit_purchase_orders (user_id, created_at desc);

create table if not exists public.api_usage_logs (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.users(id) on delete cascade,
    app_feature text not null,
    workflow_name text,
    provider_used text not null,
    model_name text,
    input_tokens integer not null default 0,
    output_tokens integer not null default 0,
    request_units numeric(18,6) not null default 0,
    is_successful boolean not null default true,
    wholesale_cost numeric(18,8) not null,
    retail_cost numeric(18,8) not null,
    external_request_id text,
    metadata_json jsonb not null default '{}'::jsonb,

```



```

    created_at timestamptz not null default now()
);

create index if not exists idx_api_usage_logs_user_id
on public.api_usage_logs (user_id, created_at desc);

create table if not exists public.credit_ledger (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.users(id) on delete cascade,
    transaction_type text not null,
    amount numeric(18,6) not null,
    reference_type text not null,
    reference_id text not null,
    description text not null,
    metadata_json jsonb not null default '{}'::jsonb,
    idempotency_key text not null unique,
    created_at timestamptz not null default now(),
    constraint chk_credit_ledger_transaction_type check (
        transaction_type in ('deposit', 'deduction', 'refund', 'adjustment', 'reversal', 'reserve',
'release')
    ),
    constraint chk_credit_ledger_nonzero_amount check (amount <> 0)
);

create index if not exists idx_credit_ledger_user_id
on public.credit_ledger (user_id, created_at desc);

create table if not exists public.stripe_webhook_events (
    stripe_event_id text primary key,
    event_type text not null,
    livemode boolean not null default false,
    api_version text,
    object_id text,
    processing_status text not null default 'received',
    processing_attempts integer not null default 0,
    last_error text,
    payload_json jsonb not null default '{}'::jsonb,
    received_at timestamptz not null default now(),
    processed_at timestamptz,
    constraint chk_stripe_webhook_processing_status check (
        processing_status in ('received', 'processing', 'processed', 'failed', 'ignored')
    )
);

create table if not exists public.swarm_runs (
    id uuid primary key default gen_random_uuid(),
    user_id uuid not null references public.users(id) on delete cascade,
    swarm_type text not null,
    status text not null default 'queued',
    batch_size integer not null default 0,
    search_radius_miles numeric(10,2),
    result_cap integer,
    estimated_wholesale_cost numeric(18,8) not null default 0,
    estimated_retail_cost numeric(18,8) not null default 0,
    actual_wholesale_cost numeric(18,8) not null default 0,
    actual_retail_cost numeric(18,8) not null default 0,
    credits_reserved numeric(18,6) not null default 0,
    credits_finalized numeric(18,6) not null default 0,
    leads_found integer not null default 0,

```

```

qualified_leads integer not null default 0,
metadata_json jsonb not null default '{}':jsonb,
created_at timestampz not null default now(),
updated_at timestampz not null default now(),
completed_at timestampz,
constraint chk_swarm_type check (swarm_type in ('market_swarm', 'vendor_swarm')),
constraint chk_swarm_status check (status in ('queued', 'running', 'completed', 'failed',
'canceled'))
);

create index if not exists idx_swarm_runs_user_created
on public.swarm_runs (user_id, created_at desc);

create table if not exists public.swarm_run_steps (
id uuid primary key default gen_random_uuid(),
swarm_run_id uuid not null references public.swarm_runs(id) on delete cascade,
step_name text not null,
provider_used text not null,
model_name text,
status text not null default 'queued',
input_tokens integer not null default 0,
output_tokens integer not null default 0,
request_units numeric(18,6) not null default 0,
wholesale_cost numeric(18,8) not null default 0,
retail_cost numeric(18,8) not null default 0,
external_request_id text,
metadata_json jsonb not null default '{}':jsonb,
created_at timestampz not null default now(),
constraint chk_swarm_step_status check (status in ('queued', 'running', 'completed', 'failed',
'skipped'))
);

create index if not exists idx_swarm_run_steps_run_id
on public.swarm_run_steps (swarm_run_id, created_at desc);

create table if not exists public.swarm_leads (
id uuid primary key default gen_random_uuid(),
swarm_run_id uuid not null references public.swarm_runs(id) on delete cascade,
lead_type text not null,
name text,
business_name text,
address text,
city text,
state text,
postal_code text,
phone text,
email text,
source_provider text,
status text not null default 'new',
score numeric(10,2),
metadata_json jsonb not null default '{}':jsonb,
created_at timestampz not null default now(),
constraint chk_swarm_lead_status check (status in ('new', 'qualified', 'rejected', 'contacted',
'archived'))
);

create index if not exists idx_swarm_leads_run_id
on public.swarm_leads (swarm_run_id, created_at desc);

```

```

alter table public.credit_packages enable row level security;
alter table public.credit_ledger enable row level security;
alter table public.api_usage_logs enable row level security;
alter table public.credit_purchase_orders enable row level security;
alter table public.pricing_configs enable row level security;
alter table public.stripe_webhook_events enable row level security;
alter table public.swarm_runs enable row level security;
alter table public.swarm_run_steps enable row level security;
alter table public.swarm_leads enable row level security;

drop policy if exists credit_packages_select_active on public.credit_packages;
create policy credit_packages_select_active
on public.credit_packages for select using (is_active = true);

drop policy if exists credit_ledger_select_own on public.credit_ledger;
create policy credit_ledger_select_own
on public.credit_ledger for select using (auth.uid() = user_id);

drop policy if exists api_usage_logs_select_own on public.api_usage_logs;
create policy api_usage_logs_select_own
on public.api_usage_logs for select using (auth.uid() = user_id);

drop policy if exists credit_purchase_orders_select_own on public.credit_purchase_orders;
create policy credit_purchase_orders_select_own
on public.credit_purchase_orders for select using (auth.uid() = user_id);

drop policy if exists swarm_runs_select_own on public.swarm_runs;
create policy swarm_runs_select_own
on public.swarm_runs for select using (auth.uid() = user_id);

drop policy if exists swarm_run_steps_select_own on public.swarm_run_steps;
create policy swarm_run_steps_select_own
on public.swarm_run_steps for select using (
  exists (
    select 1 from public.swarm_runs sr
    where sr.id = swarm_run_id and sr.user_id = auth.uid()
  )
);

drop policy if exists swarm_leads_select_own on public.swarm_leads;
create policy swarm_leads_select_own
on public.swarm_leads for select using (
  exists (
    select 1 from public.swarm_runs sr
    where sr.id = swarm_run_id and sr.user_id = auth.uid()
  )
);

insert into public.credit_packages (code, name, price_usd, credits_granted, is_active, sort_order)
values
  ('starter_1000', 'Starter', 10.00, 1000, true, 10),
  ('growth_2750', 'Growth', 25.00, 2750, true, 20),
  ('pro_6000', 'Pro', 50.00, 6000, true, 30)
on conflict (code) do nothing;

```

Appendix B. Core RPC / Database Functions

Use these functions as the only approved balance mutation paths.

```
create or replace function public.recalculate_user_credit_balance(p_user_id uuid)
returns numeric
language plpgsql
security definer
as $$
declare
    v_total numeric(18,6);
begin
    select coalesce(sum(amount), 0)
    into v_total
    from public.credit_ledger
    where user_id = p_user_id;

    update public.users
    set available_credits = v_total
    where id = p_user_id;

    return v_total;
end;
$$;

create or replace function public.deposit_credits(
    p_user_id uuid,
    p_amount numeric,
    p_reference_type text,
    p_reference_id text,
    p_description text,
    p_idempotency_key text,
    p_metadata_json jsonb default '{}'::jsonb
)
returns boolean
language plpgsql
security definer
as $$
declare
    v_existing uuid;
begin
    if p_amount <= 0 then
        raise exception 'deposit amount must be positive';
    end if;

    select id into v_existing
    from public.credit_ledger
    where idempotency_key = p_idempotency_key;

    if v_existing is not null then
        return false;
    end if;

    perform 1 from public.users
    where id = p_user_id
    for update;

    insert into public.credit_ledger (
```

```

        user_id, transaction_type, amount, reference_type, reference_id,
        description, metadata_json, idempotency_key
    ) values (
        p_user_id, 'deposit', p_amount, p_reference_type, p_reference_id,
        p_description, coalesce(p_metadata_json, '{} '::jsonb), p_idempotency_key
    );

    update public.users
    set available_credits = available_credits + p_amount
    where id = p_user_id;

    return true;
end;
$$;

create or replace function public.deduct_credits(
    p_user_id uuid,
    p_amount numeric,
    p_reference_type text,
    p_reference_id text,
    p_description text,
    p_idempotency_key text,
    p_metadata_json jsonb default '{} '::jsonb
)
returns boolean
language plpgsql
security definer
as $$
declare
    v_balance numeric(18,6);
    v_existing uuid;
begin
    if p_amount <= 0 then
        raise exception 'deduction amount must be positive';
    end if;

    select id into v_existing
    from public.credit_ledger
    where idempotency_key = p_idempotency_key;

    if v_existing is not null then
        return true;
    end if;

    select available_credits
    into v_balance
    from public.users
    where id = p_user_id
    for update;

    if v_balance is null then
        raise exception 'user not found';
    end if;

    if v_balance < p_amount then
        return false;
    end if;

```

```

insert into public.credit_ledger (
    user_id, transaction_type, amount, reference_type, reference_id,
    description, metadata_json, idempotency_key
) values (
    p_user_id, 'deduction', -p_amount, p_reference_type, p_reference_id,
    p_description, coalesce(p_metadata_json, '{} '::jsonb), p_idempotency_key
);

update public.users
set available_credits = available_credits - p_amount
where id = p_user_id;

return true;
end;
$$;

create or replace function public.mark_stripe_webhook_processed(
    p_stripe_event_id text,
    p_status text,
    p_last_error text default null
)
returns void
language plpgsql
security definer
as $$
begin
    update public.stripe_webhook_events
    set
        processing_status = p_status,
        processed_at = case when p_status = 'processed' then now() else processed_at end,
        processing_attempts = processing_attempts + 1,
        last_error = p_last_error
    where stripe_event_id = p_stripe_event_id;
end;
$$;

```

Optional swarm reserve-then-settle guidance

- For very expensive or long-running swarm runs, reserve estimated credits first, write reserve and release ledger entries if you want stronger spend control, then settle the final deduction when the run completes.
- If you do not reserve, at minimum write the swarm_runs estimated and actual cost fields and block large runs when balance is clearly insufficient.

Appendix C. AI Agent Implementation Notes

- Use the schema in Appendix A as the single source of truth.
- Use the functions in Appendix B as the only allowed balance mutation path.
- Do not bypass the ledger from application code.
- Do not let Stripe success URLs update balance.
- Implement webhook verification using the raw request body.

- Treat Market Swarm and Vendor Swarm as premium credit-gated workflows with parent-child cost tracking.